

SQL & PL/SQL

Interview Questions & Answers

A Complete Guide — Beginner to Expert Level

With clear explanations and practical code examples for every topic

#	Section	Q&A Count
1	SQL FUNDAMENTALS (Beginner)	15
2	SQL INTERMEDIATE (Joins, Subqueries, Constraints, Views)	15
3	SQL ADVANCED / EXPERT (Window Functions, CTEs, Transactions, Tuning)	15
4	PL/SQL FUNDAMENTALS (Beginner)	12
5	PL/SQL INTERMEDIATE (Cursors, Exceptions, Procedures & Functions)	15
6	PL/SQL ADVANCED (Packages, Triggers, Collections, Bulk & Dynamic SQL)	15
7	EXPERT LEVEL (Performance, Architecture & Best Practices)	13
	TOTAL	100

Covers Core SQL • Joins & Subqueries • Window Functions
Transactions & Tuning • PL/SQL Blocks & Cursors
Packages, Triggers & Collections • Bulk Ops & Performance Tuning

Table of Contents

1. SQL FUNDAMENTALS (Beginner) (15 questions)
2. SQL INTERMEDIATE (Joins, Subqueries, Constraints, Views) (15 questions)
3. SQL ADVANCED / EXPERT (Window Functions, CTEs, Transactions, Tuning) (15 questions)
4. PL/SQL FUNDAMENTALS (Beginner) (12 questions)
5. PL/SQL INTERMEDIATE (Cursors, Exceptions, Procedures & Functions) (15 questions)
6. PL/SQL ADVANCED (Packages, Triggers, Collections, Bulk & Dynamic SQL) (15 questions)
7. EXPERT LEVEL (Performance, Architecture & Best Practices) (13 questions)

How to use this guide: Each question includes a concise, interview-ready answer and, where useful, a short working SQL / PL-SQL example. Work through the parts in order for a structured beginner-to-expert refresher, or jump directly to the topic you need.

PART 1 — SQL FUNDAMENTALS (Beginner)

Q1. What is SQL?

SQL (Structured Query Language) is a standard language used to create, read, update, and manage data stored in a relational database. It is declarative - you describe what result you want, and the database engine decides how to get it.

Q2. What are the different categories of SQL commands?

SQL commands are grouped into five categories: DDL (Data Definition Language) defines structure - CREATE, ALTER, DROP, TRUNCATE; DML (Data Manipulation Language) changes data - INSERT, UPDATE, DELETE, MERGE; DQL (Data Query Language) reads data - SELECT; DCL (Data Control Language) manages permissions - GRANT, REVOKE; TCL (Transaction Control Language) manages transactions - COMMIT, ROLLBACK, SAVEPOINT.

Q3. What is the difference between CHAR and VARCHAR2?

CHAR is a fixed-length data type; it always reserves the declared number of bytes and pads unused space with blanks. VARCHAR2 is variable-length; it stores only the characters supplied and uses only the space it needs, which usually makes it more storage-efficient.

```
-- CHAR(10) storing 'SQL' uses 10 bytes: 'SQL      '  
-- VARCHAR2(10) storing 'SQL' uses only 3 bytes: 'SQL'  
CREATE TABLE demo (  
    code CHAR(5),  
    name VARCHAR2(50)  
);
```

Q4. What is a Primary Key? How is it different from a Unique Key?

A Primary Key uniquely identifies each row in a table; it does not allow NULLs and a table can have only one primary key (which may be composite). A Unique Key also enforces uniqueness but allows one NULL value (Oracle) and a table can have multiple unique keys.

```
CREATE TABLE employees (  
    emp_id    NUMBER PRIMARY KEY,  
    email     VARCHAR2(100) UNIQUE,  
    ssn       VARCHAR2(11) UNIQUE  
);
```

Q5. What is a Foreign Key?

A Foreign Key is a column (or set of columns) in one table that references the Primary Key or a Unique Key of another table. It enforces referential integrity, ensuring that a value inserted in the child table already exists in the parent table.

```
CREATE TABLE orders (  
  order_id  NUMBER PRIMARY KEY,  
  emp_id    NUMBER,  
  CONSTRAINT fk_emp FOREIGN KEY (emp_id)  
    REFERENCES employees(emp_id)  
);
```

Q6. What is the difference between WHERE and HAVING?

WHERE filters individual rows before any grouping takes place and cannot use aggregate functions.

HAVING filters groups after GROUP BY has been applied and is typically used with aggregate functions like SUM, COUNT, or AVG.

```
SELECT dept_id, COUNT(*) AS emp_count  
FROM employees  
WHERE status = 'ACTIVE'  
GROUP BY dept_id  
HAVING COUNT(*) > 5;
```

Q7. What is the logical order of execution of a SELECT statement?

Although we write SELECT first, the engine logically evaluates clauses in this order: FROM, JOIN, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, then FETCH/LIMIT. Understanding this order explains why you cannot use a SELECT alias in a WHERE clause but can use it in ORDER BY.

Q8. What is the difference between DELETE, TRUNCATE, and DROP?

DELETE is a DML command that removes rows one at a time (can be filtered with WHERE, is logged, and can be rolled back). TRUNCATE is a DDL command that removes all rows at once, resets storage, is minimally logged, and is very fast, but cannot normally be rolled back. DROP removes the entire table structure along with its data from the database.

```
DELETE FROM orders WHERE order_date < DATE '2020-01-01';  
TRUNCATE TABLE temp_logs;  
DROP TABLE old_archive;
```

Q9. How does NULL behave in SQL?

NULL represents an unknown or missing value. It is never equal to anything, not even another NULL, so comparisons like value = NULL always return unknown/false; you must use IS NULL or IS NOT NULL.

Aggregate functions (except COUNT(*)) ignore NULLs, and any arithmetic with NULL yields NULL.

```
SELECT * FROM employees WHERE commission IS NULL;  
SELECT NVL(commission, 0) FROM employees; -- replace NULL with 0
```

Q10. What is the difference between IN and EXISTS?

IN compares a value against a list or subquery result and materializes the full list before comparing; it can behave unpredictably if the list contains NULLs. EXISTS checks only whether a subquery returns at least one row and stops as soon as a match is found, which is often faster for large correlated subqueries.

```
-- IN
SELECT * FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE location='NY');

-- EXISTS
SELECT * FROM employees e
WHERE EXISTS (SELECT 1 FROM departments d
              WHERE d.dept_id = e.dept_id AND d.location='NY');
```

Q11. What is the difference between UNION and UNION ALL?

UNION combines the result sets of two or more queries and removes duplicate rows, which requires an internal sort/distinct step. UNION ALL simply concatenates the result sets without removing duplicates, making it faster when you know duplicates are not a concern.

```
SELECT city FROM customers
UNION
SELECT city FROM suppliers; -- duplicates removed

SELECT city FROM customers
UNION ALL
SELECT city FROM suppliers; -- duplicates kept
```

Q12. What are aggregate functions? Give examples.

Aggregate functions perform a calculation across a set of rows and return a single summary value. Common examples are COUNT, SUM, AVG, MIN, and MAX. They are usually used together with GROUP BY to summarize data per group.

```
SELECT dept_id,
       COUNT(*)      AS headcount,
       AVG(salary)    AS avg_salary,
       MAX(salary)    AS top_salary
FROM employees
GROUP BY dept_id;
```

Q13. What is the difference between COUNT(*), COUNT(1), and COUNT(column)?

COUNT(*) counts all rows including those with NULLs in every column. COUNT(1) behaves the same as COUNT(*) in modern optimizers since the constant is not evaluated per column - both count rows, not values. COUNT(column) counts only the rows where that specific column is NOT NULL.

```
SELECT COUNT(*)      AS total_rows,
       COUNT(commission) AS rows_with_commission
FROM employees;
```

Q14. What is a composite key?

A composite key (or compound key) is a primary key made up of two or more columns that together uniquely identify a row, used when no single column is unique enough on its own, such as an order line table keyed by (order_id, line_number).

```
CREATE TABLE order_items (  
  order_id    NUMBER,  
  line_no     NUMBER,  
  product_id  NUMBER,  
  qty         NUMBER,  
  PRIMARY KEY (order_id, line_no)  
);
```

Q15. What is the difference between DISTINCT and GROUP BY?

DISTINCT simply removes duplicate rows from the result set based on the selected columns. GROUP BY groups rows that share the same values and is required whenever you want to apply an aggregate function per group; it can also be used purely to get unique combinations, similar to DISTINCT, but is more powerful when aggregation is needed.

```
SELECT DISTINCT dept_id FROM employees;  
  
SELECT dept_id, COUNT(*) FROM employees GROUP BY dept_id;
```

PART 2 — SQL INTERMEDIATE (Joins, Subqueries, Constraints, Views)

Q16. What are the different types of JOINS?

INNER JOIN returns only matching rows from both tables. LEFT (OUTER) JOIN returns all rows from the left table plus matches from the right, with NULLs where there is no match. RIGHT (OUTER) JOIN is the mirror image. FULL OUTER JOIN returns all rows from both sides, matched where possible. SELF JOIN joins a table to itself. CROSS JOIN returns the Cartesian product of two tables.

```
SELECT e.emp_name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;

SELECT e.emp_name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
```

Q17. What is a self-join and when do you use it?

A self-join joins a table to itself using two aliases, typically used to compare rows within the same table, such as finding an employee's manager who is also stored in the employees table.

```
SELECT e.emp_name AS employee, m.emp_name AS manager
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id;
```

Q18. What is the difference between a subquery and a correlated subquery?

A regular subquery executes once, independently of the outer query, and its result is used by the outer query. A correlated subquery references a column from the outer query, so it executes once per row processed by the outer query, effectively acting like a row-by-row filter.

```
-- Simple subquery (runs once)
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- Correlated subquery (runs per row)
SELECT * FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees
               WHERE dept_id = e.dept_id);
```

Q19. When would you use a subquery versus a JOIN?

Use a JOIN when you need columns from both tables in the final result set, since joins are generally optimized well and are easy to read. Use a subquery when you only need to filter or check existence against another table without returning its columns, or when the logic is naturally expressed as a nested check (e.g., EXISTS, IN, or a scalar comparison).

Q20. What are the SQL set operators?

UNION combines results removing duplicates; UNION ALL combines without removing duplicates; INTERSECT returns only rows common to both queries; MINUS (called EXCEPT in some databases) returns rows in the first query that are not present in the second. All set operators require the same number of columns with compatible data types in each query.

```
SELECT emp_id FROM current_employees
INTERSECT
SELECT emp_id FROM training_attendees;

SELECT emp_id FROM current_employees
MINUS
SELECT emp_id FROM training_attendees;
```

Q21. What is a Cartesian product / CROSS JOIN?

A Cartesian product pairs every row of one table with every row of another, producing rows equal to the product of both row counts. It happens intentionally with CROSS JOIN, or accidentally when a join condition is missing or incorrect.

```
SELECT c.color, s.size
FROM colors c
CROSS JOIN sizes s; -- every color/size combination
```

Q22. Explain the main types of constraints in SQL.

NOT NULL disallows missing values in a column; UNIQUE ensures all values in a column are distinct; CHECK enforces a custom condition on the data; DEFAULT supplies a value when none is given; PRIMARY KEY combines NOT NULL and UNIQUE to identify each row; FOREIGN KEY enforces that values match a related table's key.

```
CREATE TABLE products (
  product_id NUMBER PRIMARY KEY,
  price       NUMBER CHECK (price > 0),
  status      VARCHAR2(10) DEFAULT 'ACTIVE' NOT NULL
);
```

Q23. What is referential integrity, and what does ON DELETE CASCADE do?

Referential integrity guarantees that a foreign key value in a child table always corresponds to an existing primary key value in the parent table, preventing orphaned rows. ON DELETE CASCADE is an option on the foreign key that automatically deletes matching child rows whenever the referenced parent row is deleted.

```
CREATE TABLE order_items (
  order_id NUMBER,
  product_id NUMBER,
  CONSTRAINT fk_order FOREIGN KEY (order_id)
    REFERENCES orders(order_id) ON DELETE CASCADE
);
```


Q24. What is a view, and what is the difference between a simple and a complex view?

A view is a stored, named SELECT statement that behaves like a virtual table; it does not store data itself (except materialized views). A simple view is based on a single table with no functions, joins, or grouping and is generally updatable. A complex view involves joins, aggregate functions, or GROUP BY, and is often read-only or only partially updatable.

```
CREATE VIEW high_earners AS
SELECT emp_id, emp_name, salary
FROM employees
WHERE salary > 100000;
```

Q25. What is a materialized view, and how does it differ from a normal view?

A normal view stores only the query definition and recomputes results every time it is queried. A materialized view actually stores the query result physically on disk, so reads are fast, but the data can become stale and needs to be refreshed periodically (on demand, on commit, or on a schedule) to reflect changes in the underlying tables.

```
CREATE MATERIALIZED VIEW mv_sales_summary
REFRESH FAST ON COMMIT
AS
SELECT region, SUM(amount) AS total_sales
FROM sales
GROUP BY region;
```

Q26. What is database normalization? Explain 1NF, 2NF, and 3NF with an example.

Normalization organizes data to reduce redundancy and avoid update anomalies. 1NF requires atomic column values with no repeating groups. 2NF requires 1NF plus every non-key column depends on the whole primary key, not just part of a composite key. 3NF requires 2NF plus no non-key column depends on another non-key column (no transitive dependency).

```
-- Violates 3NF: dept_name depends on dept_id, not directly on emp_id
employees(emp_id, emp_name, dept_id, dept_name)

-- Normalized to 3NF:
employees(emp_id, emp_name, dept_id)
departments(dept_id, dept_name)
```

Q27. What is denormalization and when is it used?

Denormalization intentionally introduces redundancy (e.g., duplicating a column, pre-joining tables) to reduce the number of joins and improve read performance. It is typically used in reporting or data-warehouse systems where read speed matters more than write efficiency or storage space.

Q28. What is an index, and what are the common types?

An index is a separate data structure that lets the database find rows quickly without scanning the whole table, at the cost of extra storage and slower writes. Common types include B-tree indexes (default, good for high-cardinality columns and range queries), bitmap indexes (good for low-cardinality columns, common in data warehouses), unique indexes (enforce uniqueness), and composite indexes (span multiple columns).

```
CREATE INDEX idx_emp_dept ON employees(dept_id);
CREATE UNIQUE INDEX idx_emp_email ON employees(email);
CREATE INDEX idx_emp_dept_status ON employees(dept_id, status);
```

Q29. When should you avoid creating an index?

Avoid indexing columns that are rarely used in WHERE/JOIN/ORDER BY clauses, columns with very low cardinality unless using a bitmap index, small tables that fit in a few blocks, and tables with heavy INSERT/UPDATE/DELETE traffic where index maintenance overhead outweighs read benefits.

Q30. What is the difference between a clustered structure and a regular heap table in Oracle?

A standard Oracle table is stored as a heap where rows have no guaranteed physical order. An Index-Organized Table (IOT) stores the actual row data within the B-tree index itself, ordered by the primary key, which speeds up range scans and primary-key lookups but can be less efficient for full table scans or secondary-key access.

```
CREATE TABLE lookup_codes (
  code_id NUMBER PRIMARY KEY,
  description VARCHAR2(100)
) ORGANIZATION INDEX;
```

PART 3 — SQL ADVANCED / EXPERT (Window Functions, CTEs, Transactions, Tuning)

Q31. What are window (analytic) functions?

Window functions perform a calculation across a set of related rows (a 'window') without collapsing the result into a single row, unlike GROUP BY aggregates. They use the OVER() clause, optionally with PARTITION BY to define groups and ORDER BY to define row order within each group.

```
SELECT emp_name, dept_id, salary,  
       AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg  
FROM employees;
```

Q32. What is the difference between RANK, DENSE_RANK, and ROW_NUMBER?

ROW_NUMBER assigns a unique sequential number to every row, even if values tie. RANK assigns the same rank to tied rows but skips subsequent rank numbers (1,2,2,4). DENSE_RANK also assigns the same rank to ties but does not skip numbers (1,2,2,3).

```
SELECT emp_name, salary,  
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn,  
       RANK()       OVER (ORDER BY salary DESC) AS rnk,  
       DENSE_RANK() OVER (ORDER BY salary DESC) AS drnk  
FROM employees;
```

Q33. What are LEAD and LAG used for?

LEAD and LAG are window functions that let you access a value from a following or preceding row respectively, without a self-join. They are commonly used to compare a row to the previous or next row, e.g., calculating month-over-month change.

```
SELECT month, revenue,  
       LAG(revenue) OVER (ORDER BY month) AS prev_month,  
       LEAD(revenue) OVER (ORDER BY month) AS next_month  
FROM monthly_sales;
```

Q34. What is a CTE (Common Table Expression), and what is a recursive CTE?

A CTE, defined with the WITH clause, is a named temporary result set that exists only for the duration of the query, improving readability by breaking complex queries into logical steps. A recursive CTE references itself to walk hierarchical or graph-like data, such as an org chart, by combining an anchor member with a recursive member.

```
WITH org_chart (emp_id, emp_name, manager_id, lvl) AS (  
  SELECT emp_id, emp_name, manager_id, 1  
  FROM employees WHERE manager_id IS NULL  
  UNION ALL  
  SELECT e.emp_id, e.emp_name, e.manager_id, oc.lvl + 1  
  FROM employees e  
  JOIN org_chart oc ON e.manager_id = oc.emp_id  
)  
SELECT * FROM org_chart;
```

Q35. What is the difference between a CTE and a view?

A CTE is temporary and exists only within the single statement that defines it, cannot be indexed, and is not stored in the data dictionary. A view is a persistent, named database object that can be reused across many queries, secured with grants, and in some cases indexed via a materialized view.

Q36. Explain the ACID properties of a transaction.

Atomicity means a transaction's operations either all succeed or all fail together. Consistency means a transaction moves the database from one valid state to another, respecting all constraints. Isolation means concurrent transactions do not interfere with each other's intermediate states. Durability means once a transaction is committed, its changes survive even a system crash.

Q37. What do COMMIT, ROLLBACK, and SAVEPOINT do?

COMMIT permanently saves all changes made in the current transaction. ROLLBACK undoes changes made since the last commit (or to a savepoint). SAVEPOINT marks an intermediate point within a transaction that you can roll back to without undoing the entire transaction.

```
UPDATE accounts SET balance = balance - 100 WHERE acct_id = 1;  
SAVEPOINT after_debit;  
UPDATE accounts SET balance = balance + 100 WHERE acct_id = 2;  
-- if something goes wrong:  
ROLLBACK TO after_debit;  
COMMIT;
```

Q38. What are transaction isolation levels?

Isolation levels control how much one transaction can see of another's uncommitted changes. READ UNCOMMITTED allows dirty reads; READ COMMITTED (Oracle's default) only sees committed data; REPEATABLE READ prevents non-repeatable reads within a transaction; SERIALIZABLE is the strictest, making transactions behave as if executed one at a time, preventing phantom reads at the cost of concurrency.

Q39. What is a deadlock, and how can it be avoided?

A deadlock occurs when two or more transactions each hold a lock the other needs, so neither can proceed. Oracle automatically detects deadlocks and rolls back one of the transactions. To avoid deadlocks, always access tables/rows in a consistent order across transactions, keep transactions short, and avoid unnecessary locking.

Q40. What is an execution plan, and how do you use EXPLAIN PLAN?

An execution plan shows the sequence of steps (full scan, index access, join method, etc.) the optimizer chooses to execute a query, along with estimated cost and cardinality. Reviewing it helps identify full table scans on large tables, missing indexes, or inefficient join orders.

```
EXPLAIN PLAN FOR
SELECT * FROM employees WHERE dept_id = 10;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Q41. What are common ways to tune a slow-running query?

Check the execution plan for full table scans on large tables and add appropriate indexes; ensure statistics are up to date; avoid functions on indexed columns in WHERE clauses; replace SELECT * with only needed columns; use EXISTS instead of IN for large subqueries where appropriate; use bind variables to enable plan reuse; and consider partitioning very large tables.

Q42. What is table partitioning and what types exist in Oracle?

Partitioning splits a large table into smaller, more manageable physical pieces while it still appears as one logical table to queries. Common types are RANGE (e.g., by date), LIST (by discrete values), HASH (evenly distributes rows), and COMPOSITE (combines two methods, such as RANGE-HASH), which can dramatically improve performance and maintenance for very large tables.

```
CREATE TABLE sales (
  sale_id NUMBER,
  sale_date DATE,
  amount NUMBER
)
PARTITION BY RANGE (sale_date) (
  PARTITION p_2024 VALUES LESS THAN (DATE '2025-01-01'),
  PARTITION p_2025 VALUES LESS THAN (DATE '2026-01-01')
);
```

Q43. What is the performance difference between EXISTS and IN with large subqueries?

EXISTS typically performs better with large or correlated subqueries because it can stop scanning as soon as one matching row is found and often uses a semi-join internally. IN materializes the entire subquery result first, which can be expensive if the list is large, though modern optimizers frequently transform IN into a semi-join automatically, narrowing the practical difference.

Q44. What are PIVOT and UNPIVOT used for?

PIVOT rotates row data into columns, useful for turning long-format data into a cross-tab report. UNPIVOT does the reverse, turning columns back into rows.

```
SELECT * FROM (  
    SELECT dept_id, quarter, amount FROM sales  
)  
PIVOT (  
    SUM(amount) FOR quarter IN ('Q1','Q2','Q3','Q4')  
);
```

Q45. What is a hierarchical query, and how does CONNECT BY work in Oracle?

A hierarchical query retrieves data that has a parent-child relationship, such as an org chart or a bill of materials. Oracle's CONNECT BY clause, together with START WITH, walks the hierarchy from a root row down (or up) through related rows, and LEVEL returns the depth of each row.

```
SELECT emp_name, LEVEL  
FROM employees  
START WITH manager_id IS NULL  
CONNECT BY PRIOR emp_id = manager_id;
```

PART 4 — PL/SQL FUNDAMENTALS (Beginner)

Q46. What is PL/SQL and how does it differ from SQL?

PL/SQL (Procedural Language/SQL) is Oracle's procedural extension to SQL. While SQL is declarative and set-based (operating on whole result sets), PL/SQL adds procedural constructs such as variables, loops, conditionals, exception handling, and reusable program units (procedures, functions, packages), letting you embed SQL inside procedural logic.

Q47. What is the basic block structure of PL/SQL?

Every PL/SQL block has up to four sections: an optional DECLARE section for variables and cursors, a mandatory BEGIN section containing the executable statements, an optional EXCEPTION section to handle errors, and a mandatory END to close the block.

```
DECLARE
    v_name VARCHAR2(50);
BEGIN
    SELECT emp_name INTO v_name FROM employees WHERE emp_id = 100;
    DBMS_OUTPUT.PUT_LINE(v_name);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employee found');
END;
/
```

Q48. What are the main categories of PL/SQL data types?

Scalar types hold a single value (NUMBER, VARCHAR2, DATE, BOOLEAN). Composite types hold multiple values (RECORD, TABLE/collections). Reference types hold pointers to other items (REF CURSOR). LOB types hold large objects (CLOB, BLOB, NCLOB, BFILE).

Q49. What are %TYPE and %ROWTYPE?

%TYPE declares a variable with the exact same data type as a specified column (or another variable), so it automatically stays in sync if the column definition changes. %ROWTYPE declares a record variable that mirrors an entire row of a table, view, or cursor, with one field per column.

```
DECLARE
    v_salary    employees.salary%TYPE;
    v_emp_row   employees%ROWTYPE;
BEGIN
    SELECT * INTO v_emp_row FROM employees WHERE emp_id = 100;
    v_salary := v_emp_row.salary;
END;
/
```

Q50. How do you declare variables and constants in PL/SQL?

Variables are declared in the DECLARE section with a name and data type, optionally with a default value using := or DEFAULT. Constants are declared the same way but with the CONSTANT keyword, and must be assigned a value at declaration since they cannot be changed afterward.

```
DECLARE
  v_counter    NUMBER := 0;
  c_tax_rate    CONSTANT NUMBER := 0.18;
BEGIN
  v_counter := v_counter + 1;
END;
/
```

Q51. How do conditional statements work in PL/SQL?

PL/SQL supports IF-THEN, IF-THEN-ELSE, and IF-THEN-ELSIF-ELSE for branching logic, as well as a CASE statement/expression for evaluating multiple discrete conditions cleanly.

```
IF v_salary > 100000 THEN
  v_grade := 'A';
ELSIF v_salary > 50000 THEN
  v_grade := 'B';
ELSE
  v_grade := 'C';
END IF;
```

Q52. What looping constructs does PL/SQL provide?

The basic LOOP runs until an explicit EXIT (or EXIT WHEN) is hit. The WHILE loop checks a condition before each iteration. The FOR loop iterates over a fixed numeric range or, as a cursor FOR loop, iterates over the rows returned by a query.

```
FOR i IN 1..5 LOOP
  DBMS_OUTPUT.PUT_LINE('Iteration ' || i);
END LOOP;

WHILE v_counter < 10 LOOP
  v_counter := v_counter + 1;
END LOOP;
```

Q53. What is the difference between an anonymous block and a named block?

An anonymous block has no name, is not stored in the database, and is compiled and run once each time it is executed - useful for scripts and testing. A named block (procedure, function, package, or trigger) is compiled once, stored in the database, and can be called repeatedly by name from other programs.

Q54. What does DBMS_OUTPUT.PUT_LINE do?

DBMS_OUTPUT.PUT_LINE writes a line of text to a buffer that a client tool (like SQL*Plus or SQL Developer) can display, mainly used for debugging and simple output; it requires SET SERVEROUTPUT ON in most clients to be visible.

```
SET SERVEROUTPUT ON;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Hello from PL/SQL');
END;
/
```

Q55. What operators are available in PL/SQL?

PL/SQL supports arithmetic operators (+, -, *, /, **), comparison operators (=, !=, <, >, <=, >=, BETWEEN, LIKE, IN), logical operators (AND, OR, NOT), and the concatenation operator (||).

Q56. What is the conceptual difference between a procedure and a function in PL/SQL?

A procedure performs an action and does not have to return a value (though it can return values via OUT parameters). A function is designed to compute and return exactly one value using a RETURN statement, and can typically be called directly within a SQL expression, unlike a plain procedure.

Q57. What happens if a SELECT INTO returns zero rows or more than one row?

If a SELECT INTO statement matches zero rows, Oracle raises the NO_DATA_FOUND exception. If it matches more than one row, Oracle raises TOO_MANY_ROWS, since a SELECT INTO expects to assign exactly one row into the target variables.

```
BEGIN
    SELECT salary INTO v_salary FROM employees WHERE emp_id = 999;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee not found');
END;
/
```

PART 5 — PL/SQL INTERMEDIATE (Cursors, Exceptions, Procedures & Functions)

Q58. What is a cursor? What is the difference between implicit and explicit cursors?

A cursor is a pointer to the private memory area (context area) Oracle uses to process a SQL statement, letting you work with the result set row by row. An implicit cursor is created and managed automatically by Oracle for any single SQL DML/SELECT statement. An explicit cursor is declared and controlled manually by the developer, needed whenever a query can return multiple rows that must be processed one at a time.

```
DECLARE
  CURSOR c_emp IS SELECT emp_name FROM employees WHERE dept_id = 10;
  v_name employees.emp_name%TYPE;
BEGIN
  OPEN c_emp;
  LOOP
    FETCH c_emp INTO v_name;
    EXIT WHEN c_emp%NOTFOUND;
    DBMS_OUTPUT.PUT_LINE(v_name);
  END LOOP;
  CLOSE c_emp;
END;
/
```

Q59. What are the standard cursor attributes?

%FOUND is TRUE if the last fetch returned a row; %NOTFOUND is TRUE if the last fetch did not return a row (commonly used to exit a loop); %ROWCOUNT returns the number of rows fetched so far; %ISOPEN returns TRUE if the cursor is currently open.

Q60. What is a cursor FOR loop?

A cursor FOR loop implicitly opens a cursor, fetches each row into a record, and closes the cursor automatically once the loop finishes, eliminating the need to write explicit OPEN, FETCH, and CLOSE statements.

```
BEGIN
  FOR emp_rec IN (SELECT emp_name, salary FROM employees WHERE dept_id = 10) LOOP
    DBMS_OUTPUT.PUT_LINE(emp_rec.emp_name || ' - ' || emp_rec.salary);
  END LOOP;
END;
/
```

Q61. What is a parameterized cursor?

A parameterized cursor accepts one or more parameters, so the same cursor definition can be reused with different filter values each time it is opened, instead of writing a new cursor for every value.

```
DECLARE
  CURSOR c_emp(p_dept NUMBER) IS
    SELECT emp_name FROM employees WHERE dept_id = p_dept;
BEGIN
  FOR r IN c_emp(10) LOOP
    DBMS_OUTPUT.PUT_LINE(r.emp_name);
  END LOOP;
END;
/
```

Q62. How does exception handling work in PL/SQL, and what are predefined exceptions?

PL/SQL uses an EXCEPTION section within a block to catch and handle runtime errors, preventing the program from crashing. Predefined exceptions correspond to common Oracle errors and already have names, such as NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE, VALUE_ERROR, and DUP_VAL_ON_INDEX.

```
BEGIN
  v_result := 10 / v_divisor;
EXCEPTION
  WHEN ZERO_DIVIDE THEN
    DBMS_OUTPUT.PUT_LINE('Cannot divide by zero');
END;
/
```

Q63. How do you define and raise a user-defined exception?

You declare a user-defined exception in the DECLARE section using the EXCEPTION type, raise it explicitly with RAISE when a business condition is violated, and handle it in the EXCEPTION section just like a predefined exception.

```
DECLARE
  e_invalid_salary EXCEPTION;
BEGIN
  IF v_salary < 0 THEN
    RAISE e_invalid_salary;
  END IF;
EXCEPTION
  WHEN e_invalid_salary THEN
    DBMS_OUTPUT.PUT_LINE('Salary cannot be negative');
END;
/
```

Q64. What is RAISE_APPLICATION_ERROR used for?

RAISE_APPLICATION_ERROR lets you raise a custom error with a user-defined error number (between -20000 and -20999) and a custom message, which propagates back to the calling application as a normal Oracle error - very useful inside procedures/triggers to enforce business rules.

```
IF v_balance < p_amount THEN
  RAISE_APPLICATION_ERROR(-20001, 'Insufficient balance');
END IF;
```

Q65. What is WHEN OTHERS, and why is an unqualified WHEN OTHERS considered bad practice?

WHEN OTHERS is a catch-all exception handler that traps any exception not explicitly handled elsewhere. Used alone without re-raising or logging the error (e.g., via SQLCODE/SQLERRM), it silently swallows unexpected errors, making bugs very hard to diagnose; best practice is to log the error details and then RAISE (or RAISE_APPLICATION_ERROR) so the caller still knows something failed.

```
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
    RAISE;
```

Q66. How do you create a stored procedure?

A stored procedure is created with CREATE [OR REPLACE] PROCEDURE, followed by an optional parameter list and the procedure body (declare/begin/exception/end). It is compiled once and stored in the database, ready to be called by name.

```
CREATE OR REPLACE PROCEDURE raise_salary(
  p_emp_id IN NUMBER,
  p_pct    IN NUMBER
) IS
BEGIN
  UPDATE employees
  SET salary = salary * (1 + p_pct/100)
  WHERE emp_id = p_emp_id;
END raise_salary;
/

-- Call it:
BEGIN
  raise_salary(100, 10);
END;
/
```

Q67. How do you create a function, and how is it different from a procedure?

A function is created with CREATE [OR REPLACE] FUNCTION and must include a RETURN data type and at least one RETURN statement that sends back a single value. Unlike a procedure, a function (if it has no side effects/DML restrictions) can usually be called directly inside a SQL SELECT statement.

```
CREATE OR REPLACE FUNCTION get_annual_salary(p_emp_id NUMBER)
RETURN NUMBER IS
    v_monthly NUMBER;
BEGIN
    SELECT salary INTO v_monthly FROM employees WHERE emp_id = p_emp_id;
    RETURN v_monthly * 12;
END;
/

SELECT get_annual_salary(100) FROM dual;
```

Q68. What are IN, OUT, and IN OUT parameters?

IN (the default) passes a value into the subprogram, which can be read but not modified inside it. OUT is used to pass a value back out to the caller; its initial value is ignored and it must be assigned inside the subprogram. IN OUT does both - the caller passes in a value, the subprogram can read and modify it, and the updated value is passed back.

```
CREATE OR REPLACE PROCEDURE get_emp_info(
    p_emp_id IN NUMBER,
    p_name OUT VARCHAR2,
    p_salary IN OUT NUMBER
) IS
BEGIN
    SELECT emp_name, salary INTO p_name, p_salary
    FROM employees WHERE emp_id = p_emp_id;
END;
/
```

Q69. What is overloading in PL/SQL?

Overloading lets you define multiple procedures or functions with the same name inside a package, as long as their parameter lists differ in number, order, or data type, so the compiler can determine which one to call based on the arguments supplied.

```
PACKAGE calc IS
    FUNCTION add_values(a NUMBER, b NUMBER) RETURN NUMBER;
    FUNCTION add_values(a VARCHAR2, b VARCHAR2) RETURN VARCHAR2;
END calc;
```

Q70. What is a forward declaration in PL/SQL?

A forward declaration lets you declare a subprogram's signature earlier in a PL/SQL block or package body before its full implementation appears, which is required when two subprograms call each other mutually (a call to a subprogram must be known before it is used).

```
PROCEDURE proc_b(p NUMBER); -- forward declaration

PROCEDURE proc_a(p NUMBER) IS
BEGIN
    proc_b(p); -- can call it because of forward declaration
END;

PROCEDURE proc_b(p NUMBER) IS
BEGIN
    NULL;
END;
```

Q71. Can you call a procedure directly from a SQL statement? Can you call a function?

A plain procedure cannot be called directly inside a SQL statement; it can only be called from another PL/SQL block. A function that meets certain purity rules (no DML side effects, deterministic behavior with respect to session state in most contexts) can be called directly inside SQL, such as in a SELECT list or WHERE clause.

```
SELECT emp_name, get_annual_salary(emp_id) FROM employees;
```

Q72. What is the difference between NO_DATA_FOUND and TOO_MANY_ROWS?

NO_DATA_FOUND is raised when a SELECT INTO returns zero rows. TOO_MANY_ROWS is raised when a SELECT INTO (which expects exactly one row) instead matches more than one row. Both only apply to SELECT INTO, not to explicit cursors, which simply return no rows without raising an exception.

```
BEGIN
    SELECT salary INTO v_sal FROM employees WHERE dept_id = 10;
EXCEPTION
    WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('No rows');
    WHEN TOO_MANY_ROWS THEN DBMS_OUTPUT.PUT_LINE('More than one row');
END;
/
```

PART 6 — PL/SQL ADVANCED (Packages, Triggers, Collections, Bulk & Dynamic SQL)

Q73. What is a PL/SQL package, and what is the difference between the spec and the body?

A package is a schema object that groups related procedures, functions, variables, cursors, and types into one logical unit. The package specification declares the public interface (what is visible to callers), while the package body contains the actual implementation, including any private helper subprograms not listed in the spec.

```
CREATE OR REPLACE PACKAGE emp_pkg IS
    PROCEDURE raise_salary(p_emp_id NUMBER, p_pct NUMBER);
    FUNCTION get_salary(p_emp_id NUMBER) RETURN NUMBER;
END emp_pkg;
/

CREATE OR REPLACE PACKAGE BODY emp_pkg IS
    PROCEDURE raise_salary(p_emp_id NUMBER, p_pct NUMBER) IS
    BEGIN
        UPDATE employees SET salary = salary*(1+p_pct/100)
        WHERE emp_id = p_emp_id;
    END;

    FUNCTION get_salary(p_emp_id NUMBER) RETURN NUMBER IS
        v_sal NUMBER;
    BEGIN
        SELECT salary INTO v_sal FROM employees WHERE emp_id=p_emp_id;
        RETURN v_sal;
    END;
END emp_pkg;
/
```

Q74. What are the advantages of using packages?

Packages improve modularity and organization by grouping related logic together; they support information hiding since only the spec is public; they improve performance because the whole package is loaded into memory on first use and cached for subsequent calls; and they allow package-level (global) variables that persist for the duration of a session.

Q75. What is a trigger, and what types exist?

A trigger is a stored PL/SQL block that fires automatically in response to a specified event on a table, view, schema, or database. Triggers can fire BEFORE or AFTER the triggering DML statement (INSERT, UPDATE, DELETE), and can be defined at the ROW level (once per affected row) or STATEMENT level (once per statement, regardless of rows affected).

```
CREATE OR REPLACE TRIGGER trg_before_insert_emp
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    :NEW.created_date := SYSDATE;
END;
/
```

Q76. What is a compound trigger?

A compound trigger lets you write a single trigger with up to four timing sections (BEFORE STATEMENT, BEFORE EACH ROW, AFTER EACH ROW, AFTER STATEMENT) in one body, sharing state across them. It is often used to solve the mutating table problem by collecting row-level changes and processing them once at the statement level.

```
CREATE OR REPLACE TRIGGER trg_compound
FOR UPDATE ON employees
COMPOUND TRIGGER
    BEFORE STATEMENT IS BEGIN NULL; END BEFORE STATEMENT;
    AFTER EACH ROW IS BEGIN NULL; END AFTER EACH ROW;
END trg_compound;
/
```

Q77. What do :NEW and :OLD mean in a trigger?

:OLD refers to the value of a column before the triggering DML statement (available in UPDATE and DELETE triggers), and :NEW refers to the value after the change (available in INSERT and UPDATE triggers). They are only meaningful in row-level triggers.

```
CREATE OR REPLACE TRIGGER trg_audit_salary
BEFORE UPDATE OF salary ON employees
FOR EACH ROW
BEGIN
    IF :NEW.salary != :OLD.salary THEN
        INSERT INTO salary_audit VALUES (:OLD.emp_id, :OLD.salary, :NEW.salary, SYSDATE);
    END IF;
END;
/
```

Q78. What is a mutating table error, and how do you avoid it?

A mutating table error (ORA-04091) occurs when a row-level trigger tries to query or modify the same table that is currently being changed by the triggering statement, because Oracle cannot guarantee a consistent view of a table still being modified. It can be avoided by using a compound trigger to defer the logic to the AFTER STATEMENT section, using a package-level collection to accumulate row data first, or restructuring the logic to avoid querying the same table.

Q79. What are the collection types in PL/SQL?

Associative Arrays (Index-By Tables) are key-value pairs indexed by an integer or string, not stored in the database, and do not need initializing. VARRAYs (variable-size arrays) have a fixed maximum size, keep their element order, and can be stored in database columns. Nested Tables are like unbounded arrays that can be stored in the database and support set-like operations, but must be initialized before use.

```
DECLARE
  TYPE t_assoc IS TABLE OF VARCHAR2(50) INDEX BY PLS_INTEGER;
  v_names t_assoc;
BEGIN
  v_names(1) := 'Alice';
  v_names(2) := 'Bob';
END;
/
```

Q80. What is BULK COLLECT and why use it?

BULK COLLECT fetches multiple rows from a SQL query into a PL/SQL collection in a single round trip between the SQL and PL/SQL engines, instead of fetching one row at a time in a loop. This drastically reduces context switches and improves performance for large data volumes.

```
DECLARE
  TYPE t_names IS TABLE OF employees.emp_name%TYPE;
  v_names t_names;
BEGIN
  SELECT emp_name BULK COLLECT INTO v_names
  FROM employees WHERE dept_id = 10;
END;
/
```

Q81. What is FORALL, and how does it differ from a normal loop with DML?

FORALL sends a bulk DML statement (INSERT, UPDATE, or DELETE) for an entire collection to the SQL engine in one operation, instead of looping and issuing one DML statement per row. This avoids repeated context switching between the PL/SQL and SQL engines and can be dramatically faster than a row-by-row loop for large datasets.

```
DECLARE
  TYPE t_ids IS TABLE OF NUMBER;
  v_ids t_ids := t_ids(101, 102, 103);
BEGIN
  FORALL i IN v_ids.FIRST..v_ids.LAST
    UPDATE employees SET salary = salary * 1.1 WHERE emp_id = v_ids(i);
END;
/
```

Q82. What is dynamic SQL, and how do you execute it with EXECUTE IMMEDIATE?

Dynamic SQL lets you build and run a SQL or PL/SQL statement as a string at runtime, useful when the table name, column list, or WHERE clause is not known until execution time. EXECUTE IMMEDIATE is the simplest way to run such a string, optionally binding values in with USING and returning results with INTO.

```
DECLARE
  v_sql VARCHAR2(200);
  v_count NUMBER;
BEGIN
  v_sql := 'SELECT COUNT(*) FROM employees WHERE dept_id = :1';
  EXECUTE IMMEDIATE v_sql INTO v_count USING 10;
END;
/
```

Q83. What is a REF CURSOR, and what is the difference between a strong and a weak REF CURSOR?

A REF CURSOR is a pointer to a query result set that can be passed between subprograms or back to a calling application (commonly used to return result sets from a procedure). A strong REF CURSOR is defined with a fixed return type (RETURN employees%ROWTYPE), enforcing type-checking; a weak REF CURSOR has no return type restriction and can be associated with any query, offering more flexibility but less compile-time safety.

```
CREATE OR REPLACE PROCEDURE get_employees(
  p_dept_id IN NUMBER,
  p_cursor OUT SYS_REFCURSOR
) IS
BEGIN
  OPEN p_cursor FOR
    SELECT emp_name, salary FROM employees WHERE dept_id = p_dept_id;
END;
/
```

Q84. What is a pipelined function?

A pipelined table function returns rows incrementally, one at a time (using PIPE ROW), instead of building the entire collection in memory before returning it. It can be queried directly in the FROM clause of a SQL statement like a table, which is useful for transforming data or streaming large result sets efficiently.

```
CREATE OR REPLACE FUNCTION get_even_numbers(p_max NUMBER)
RETURN sys.odcinumberlist PIPELINED IS
BEGIN
    FOR i IN 1..p_max LOOP
        IF MOD(i,2) = 0 THEN
            PIPE ROW(i);
        END IF;
    END LOOP;
    RETURN;
END;
/

SELECT * FROM TABLE(get_even_numbers(10));
```

Q85. What is an autonomous transaction?

An autonomous transaction, declared with PRAGMA AUTONOMOUS_TRANSACTION, runs as an independent transaction inside a parent transaction; it can commit or roll back its own changes without affecting the state of the calling transaction. It is commonly used for logging or auditing errors even when the main transaction later rolls back.

```
CREATE OR REPLACE PROCEDURE log_error(p_msg VARCHAR2) IS
    PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
    INSERT INTO error_log(message, log_date) VALUES (p_msg, SYSDATE);
    COMMIT;
END;
/
```

Q86. How does the state of package-level variables behave across a session?

Package-level variables are initialized the first time the package is referenced in a session and then persist in that session's UGA (User Global Area) for as long as the session lasts, retaining their values across multiple calls unless explicitly reset - useful for session-level caching, but care is needed since state does not automatically reset between calls.

Q87. What is a global temporary table, and how does it differ from a regular table?

A global temporary table has a persistent definition in the data dictionary, but its data is private to each session and automatically cleared either after each COMMIT (ON COMMIT DELETE ROWS) or at the end of the session (ON COMMIT PRESERVE ROWS). It is useful for staging intermediate data during complex processing without permanently storing it.

```
CREATE GLOBAL TEMPORARY TABLE temp_calc (  
    emp_id NUMBER,  
    computed_bonus NUMBER  
) ON COMMIT DELETE ROWS;
```

PART 7 — EXPERT LEVEL (Performance, Architecture & Best Practices)

Q88. What are the key techniques for tuning PL/SQL code performance?

Use BULK COLLECT and FORALL to minimize context switches between the SQL and PL/SQL engines; avoid row-by-row (slow-by-slow) processing wherever a set-based SQL statement can do the job instead; use bind variables to enable cursor sharing; cache frequently used lookup data in package-level collections; and avoid unnecessary commits inside loops.

Q89. What is the LIMIT clause with BULK COLLECT and why is it important?

An unrestricted BULK COLLECT fetches all matching rows into memory at once, which can exhaust PGA memory for very large result sets. Adding LIMIT n inside a loop fetches rows in manageable batches, balancing the performance benefit of bulk fetching against memory consumption.

```
DECLARE
  CURSOR c_emp IS SELECT emp_id FROM employees;
  TYPE t_ids IS TABLE OF NUMBER;
  v_ids t_ids;
BEGIN
  OPEN c_emp;
  LOOP
    FETCH c_emp BULK COLLECT INTO v_ids LIMIT 1000;
    EXIT WHEN v_ids.COUNT = 0;
    -- process v_ids batch
  END LOOP;
  CLOSE c_emp;
END;
/
```

Q90. What are bind variables and why do they matter for performance?

Bind variables are placeholders (like :emp_id) used in place of literal values in a SQL statement. Using them lets Oracle reuse the same parsed execution plan across executions with different values (cursor sharing), avoiding expensive hard parses; hard-coding literals instead causes a new plan to be parsed for every distinct value, which can severely hurt performance and consume shared pool memory.

```
-- Good: uses a bind variable, plan is reused
SELECT * FROM employees WHERE emp_id = :emp_id;

-- Bad: literal forces a new parse for every different value
SELECT * FROM employees WHERE emp_id = 12345;
```

Q91. What is the difference between a hard parse and a soft parse?

A hard parse happens when Oracle cannot find a matching cached execution plan for a SQL statement and must fully parse, optimize, and build a new plan - an expensive operation. A soft parse occurs when Oracle finds a reusable matching plan already in the shared pool (typically because bind variables were used) and can skip most of the optimization work, making it much cheaper.

Q92. When would you use a fast-refresh materialized view versus a complete refresh?

A fast refresh applies only the incremental changes captured since the last refresh (typically via a materialized view log on the base table), making it efficient for large tables with relatively small change volumes. A complete refresh re-executes the entire defining query and rebuilds the whole materialized view, which is simpler but far more expensive, and is generally used for smaller datasets or when a fast refresh is not possible (e.g., due to unsupported query constructs).

```
CREATE MATERIALIZED VIEW LOG ON sales WITH ROWID;

CREATE MATERIALIZED VIEW mv_sales AS
SELECT region, SUM(amount) AS total FROM sales GROUP BY region;

EXEC DBMS_MVIEW.REFRESH('MV_SALES', 'F'); -- fast refresh
```

Q93. What is the difference between a global and a local index on a partitioned table?

A local index is automatically partitioned to align one-to-one with the underlying table's partitions, so partition maintenance operations (like dropping a partition) do not affect other index partitions. A global index is partitioned independently of the base table (or not partitioned at all) and can span multiple table partitions, offering more flexibility for certain queries but requiring a rebuild (or UPDATE GLOBAL INDEXES) when partition maintenance operations occur.

```
CREATE INDEX idx_sales_local ON sales(sale_date) LOCAL;
CREATE INDEX idx_sales_global ON sales(customer_id) GLOBAL;
```

Q94. What are optimizer hints, and can you give examples?

Hints are comments embedded in a SQL statement that suggest a specific execution strategy to the cost-based optimizer, used sparingly when you have specific knowledge the optimizer lacks. Examples include /*+ INDEX(table idx_name) */ to force an index scan, /*+ FULL(table) */ to force a full table scan, /*+ PARALLEL(table, 4) */ to request parallel execution, and /*+ LEADING(a b) */ to control join order.

```
SELECT /*+ INDEX(e idx_emp_dept) */ *
FROM employees e
WHERE dept_id = 10;
```

Q95. What does the DETERMINISTIC keyword do on a function, and why does it matter?

DETERMINISTIC tells Oracle that a function always returns the same output for the same input values, allowing the optimizer to cache results and potentially use the function in function-based indexes or materialized views with query rewrite. Marking a genuinely non-deterministic function as DETERMINISTIC (e.g., one depending on SYSDATE) can produce incorrect cached results, so it should only be used when truly appropriate.

```
CREATE OR REPLACE FUNCTION calc_tax(p_amount NUMBER)
RETURN NUMBER DETERMINISTIC IS
BEGIN
    RETURN p_amount * 0.18;
END;
/
```

Q96. What is a context switch between the SQL and PL/SQL engines, and how do you minimize it?

PL/SQL and SQL run in two different engines; every time a PL/SQL block issues a SQL statement (or vice versa), control switches between them, which has a small but real overhead that adds up quickly in loops. It is minimized by replacing row-by-row DML/queries inside loops with set-based SQL, or by using BULK COLLECT and FORALL to batch many operations into a single context switch.

Q97. What is the PL/SQL WRAP utility used for?

The WRAP utility (or DBMS_DDL.WRAP / CREATE_WRAPPED) obfuscates PL/SQL source code into an unreadable but still executable format, used to protect proprietary business logic or intellectual property when distributing PL/SQL code to third parties, while still allowing Oracle to compile and run it normally.

```
-- Command line:  
-- wrap iname=my_package_body.sql oname=my_package_body.plb
```

Q98. How does error handling differ between a normal call and a call to a procedure with an autonomous transaction, if the main transaction later rolls back?

Because an autonomous transaction is fully independent, any COMMIT it issues is permanent and is not undone even if the calling (parent) transaction is later rolled back - this is precisely why autonomous transactions are used for error/audit logging that must survive a rollback of the main business transaction. Without PRAGMA AUTONOMOUS_TRANSACTION, any insert used for logging would be rolled back along with everything else.

Q99. What is the difference between UPDATE GLOBAL INDEXES and rebuilding an index after a partition operation?

When a partition-maintenance operation (like DROP PARTITION or TRUNCATE PARTITION) is performed, any global index on that table becomes unusable unless you specify UPDATE GLOBAL INDEXES in the same DDL statement, which incrementally maintains the global index as part of the operation - avoiding a separate, potentially expensive full index rebuild afterward.

```
ALTER TABLE sales DROP PARTITION p_2023 UPDATE GLOBAL INDEXES;
```

Q100. What is the difference between NOCOPY parameter passing and default (by-value) OUT/IN OUT parameter passing?

By default, OUT and IN OUT parameters are passed by value - Oracle uses a temporary copy and only writes it back to the actual variable once the subprogram completes successfully, which protects the caller's variable if an exception occurs but adds overhead for large data (like big collections). The NOCOPY hint asks Oracle to pass the parameter by reference instead, improving performance for large structures, at the cost of losing that rollback protection if an exception is raised mid-execution.

```
PROCEDURE process_data(p_data IN OUT NOCOPY t_large_collection) IS
BEGIN
    -- large collection is passed by reference for performance
    NULL;
END;
```
